Week 2 — HTML5

Full Teaching Content — Lectures 4–6, Tutorial, Lab

BO CDA 208 — Web Development and App Design.

HTML fundamentals → semantic HTML & accessibility → forms & modern HTML, closing with the semester-project tutorial and the multi-page lab.

Lecture 4 — HTML Fundamentals

A. Introduction to HTML

- **What is HTML?** HyperText Markup Language — a markup language, not a programming language. It describes structure and content, not logic.
- **Role of HTML in Web development** — HTML is the structure/content layer only. It says nothing about color, layout, or interactivity — that's CSS's and JavaScript's job, coming in later weeks.
- **HTML vs. CSS vs. JavaScript** — HTML = structure, CSS = presentation, JS = behavior. Keep this three-way split visible all semester; almost every later topic is "which of these three am I touching right now."
- **HTML as a markup language** — a markup language tells a browser how to present or organize information; a programming language tells a computer how to compute something. HTML has no variables, no loops, no logic.
- **HTML document vs. Web page** — the .html file is the document (what you edit); the "web page" is what the browser renders from it. Same edit/preview split introduced in Week 1's Setup discussion.
- **HTML5 and the evolution of HTML** — HTML (1991) → HTML 4.01 (1999) → XHTML (a stricter, XML-based dead end) → HTML5 (2014), which added semantic elements, native audio/video, canvas, and new input types. HTML is now a "living standard" maintained by WHATWG — continuously updated rather than released in discrete numbered versions.

B. Basic HTML Document Structure

The full boilerplate, run once end to end, then broken apart:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>

</body>
</html>
```

Element	Role
<!DOCTYPE html>	Required preamble; ensures the browser renders in standards mode.
<html>	Root element of the document; contains all other elements.
<head>	Container for metadata — nothing here is shown directly to visitors.
<title>	Text shown in the browser tab and used when bookmarking.
<body>	Everything visitors actually see.

- Comments — is never rendered; it's for developers reading the source only. Not truly hidden — visible via View Page Source.

C. HTML Elements

- **What is an element?** A start tag, content, and end tag together — the basic building block of a page.
- **Opening and closing tags** — opens, closes; forgetting the closing tag is the single most common beginner error.
- **Empty/void elements** — elements with no content to wrap, so no closing tag:
, <hr>, <img>, <input>, <meta>, <link>.
- **Nested elements** — elements placed inside other elements; nesting must not cross (close tags in the reverse order you opened them).
- **Parent and child elements** — the containing element is the parent, the contained one is the child. This vocabulary will resurface constantly once JavaScript starts walking the DOM in Week 6.
- **Block-level vs. inline elements** — conceptual introduction only: block elements (p, div, h1–h6) start on a new line and take the full available width; inline elements (span, a, strong) flow within a line of text. Full layout consequences are a CSS topic, Weeks 3–4 — here, just recognize the two categories.

D. HTML Attributes

- **What is an attribute?** A name="value" pair inside a start tag that configures or describes the element.
- **Global attributes** — available on virtually any element: id, class, title, style, data-*, lang, hidden.

Attribute	Use
id	A unique identifier for one specific element on the page — used for in-page links, CSS, and JavaScript targeting.
class	A reusable label; many elements can share the same class, and one element can carry several classes.
title	Tooltip text shown on mouse-over (covered already for images — works the same way on any element).
style	Inline CSS — brief mention only. Real styling is external stylesheets, starting Week 3; treat inline style as a last resort, not a habit.
data-*	Custom data attributes for storing extra information JavaScript can read later, e.g. data-user-id="42".

E. Text Content

Headings (<h1>–<h6>),paragraphs ( ), line breaks (
), and horizontal rules (<hr>) were introduced in the Beginning HTML material — recap briefly, then focus class time on the semantic point below.

- ** vs. ** over <b> and <i>: and signal genuine importance/emphasis to screen readers and search engines; <b> and <i> are purely visual with no such signal.

  Why semantic text elements are preferred: the visual result can look identical, but only the semantic version tells assistive technology and machines that the meaning matters, not just the appearance.

F. Hyperlinks

```
<a href="about.html">About</a>
<a href="https://example.com">External site</a>
<a href="#section2">Jump to Section 2</a>
<a href="mailto:prof@example.com">Email</a>
<a href="tel:+911234567890">Call</a>
```

- **Absolute URLs** — the full address, including https://; needed for linking off-site.
- **Relative URLs** — a path relative to the current file; used for linking between pages of the same site (see Week 1's File Paths rules).
- **Linking to sections within a page** — give the target element an id, then link to it with href="#that-id".
- **Opening links in a new tab** — target="_blank".
- **mailto: and tel: links** — open the user's email client or phone dialer directly.

  Security consideration: target="_blank" without rel="noopener noreferrer" lets the new tab's page control the original tab via window.opener — a known phishing vector called reverse tabnabbing. Always pair the two.

G. Images

<img>, src, alt, width/height, relative vs. absolute paths, and images-as-links were covered in the Beginning HTML material. Recap the pattern once, then restate the two distinct reasons alt matters: screen readers read it aloud, and it displays if the image fails to load.

H. Lists

<ul</<ol</<li< and nesting were covered previously. New this lecture:

- Description lists — <dl<, <dt> (term), <dd> (definition) — for term/definition pairs, like a glossary.

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets</dd>
</dl>
```

I. Tables

<table</<tr>/<th>/<td> and <caption> were covered previously. New this lecture:

Element / Attribute	Purpose
<thead>, <tbody>, <tfoot>	Group header rows, body rows, and footer rows — lets browsers and assistive tech distinguish the table's regions.
colspan	Merges a cell across multiple columns.
rowspan	Merges a cell across multiple rows.
scope (basic accessibility)	On a <th>, declares whether it heads a column or a row, so screen readers can announce the right header for each data cell.

End-of-lecture target: students should be able to construct a complete basic HTML document containing text → links → images → lists → tables, unaided.

Lecture 5 — Semantic HTML & Accessibility

A. Why Semantic HTML?

- **Semantic HTML** = elements whose tag name describes their own meaning (<article>, <nav>) rather than saying nothing about their content (,).

Benefits, worth naming individually rather than as one vague “it's good practice” claim:

Benefits for…	What semantic HTML gives them
Browsers	Can apply sensible default behavior and rendering without guessing.
Developers	Code that documents its own structure — easier to read six months later.
Accessibility	Screen readers can announce regions (“navigation,” “main content”) instead of a flat wall of divs.
Search engines	Can identify the actual article content vs. boilerplate navigation/ads, which affects indexing.

B. Semantic Page Structure

Element	Represents
<header>	Introductory content or navigation aids for its nearest sectioning ancestor.
<nav>	A block of navigation links.
<main>	The dominant, unique content of the document — only one per page.
<section>	A generic, thematically grouped section of content, usually with its own heading.
<article>	A self-contained piece of content — a blog post, a news story.
<aside>	Content tangentially related to the main content — a sidebar, a pull quote.
<footer>	Footer content for its nearest sectioning ancestor.

C. Understanding Their Roles

- **Page header vs. section header** — <header> is not a once-per-page element. A page can have one <header< at the top and another <heade<> inside each <article<, each scoped to its nearest sectioning ancestor.
- **Navigation areas** — not every group of links is a <nav> — reserve it for major navigation blocks (site nav, table of contents), not every link cluster.
- **Main content** — exactly one <main> per page, excluding anything repeated across pages (nav, sidebars, footers).
- **Independent articles** — the test for <article>: would this content still make sense if syndicated elsewhere on its own (an RSS feed, a different site)? If yes, it's an article.
- **Sections within a page** — <section> groups thematically related content that isn't independently distributable — usually has its own heading.
- **Complementary content** — <aside> for content related to but separable from the main flow — remove it and the main content still stands alone.
- **Footer content** — like header, can appear once per page and also once per article/section.

D. Building a Semantic Page

Typical page hierarchy:

```
<html>
 └── body
      ├── header
      ├── nav
      ├── main
      │    ├── section
      │    │    └── article
      │    └── aside
      └── footer
```

Pedagogical choice: teach semantic elements first, not <div<+CSS-everywhere. Once semantics are second nature, the rule for div/span becomes simple: div is a generic block container for when no semantic element fits; span is the same for inline content. That produces far better habits than starting from div and retrofitting semantics later.

- Choosing the appropriate semantic element and avoiding unnecessary usage are both direct consequences of that one rule.

E. Introduction to Web Accessibility

- **What is Web accessibility?** Designing and building sites usable by people with a full range of abilities, including visual, motor, auditory, and cognitive disabilities.
- **Why it matters** — it's both an ethical baseline and, for many organizations, a legal requirement; and accessible sites are frequently better for everyone (captions help in a noisy room, not just for deaf users).
- **Keyboard accessibility** — every interactive element must be reachable and operable using Tab/Shift+Tab/Enter alone, with a visible focus indicator. Semantic elements (button, a) get this for free; a does not.
- **Screen readers** — software that reads page content aloud, relying heavily on semantic structure and alt text to describe what's on screen.
- **Semantic HTML and assistive technology** — this is the payoff of Sections A–D: every semantic choice made above directly determines what a screen reader announces.

F. Alternative Text

- **Purpose of alt** — read aloud by screen readers; shown if the image fails to load.
- **Writing meaningful alt text** — describe the content or function of the image, not its appearance for its own sake — “Company logo” not “blue swirl graphic.”
- **Decorative images** — images that carry no information (a background flourish) should use alt="" (empty, not omitted) so screen readers skip them silently.
- **Common mistakes** — redundant prefixes (“image of…” — the screen reader already announces it's an image), and keyword-stuffed alt text written for search engines instead of users.

G. Forms and Accessibility

- **Importance of labels** — an input with no associated label is unusable by screen-reader users — they hear “edit text” with no indication of what to type.
- **<label>** — associates descriptive text with a form control.

```
<label for="email">Email address</label>
<input id="email" type="email">
```

- **Associating labels with controls** — either the for/id pairing above, or by wrapping the input inside the <label> itself: <label>Email <input type="email"></label>.

Full form treatment — input types, validation, submission — is Lecture 6's job; this is only the accessibility groundwork.

H. ARIA

- **What is ARIA?** Accessible Rich Internet Applications — attributes that supply extra roles, states, and properties for assistive technology when native HTML alone isn't enough.
- **Roles** — define an element's purpose (role="tab");
- **States** — an element's current condition (aria-pressed="true");
- **Properties** — descriptive metadata (aria-label="Search the site").

First rule of ARIA: use native HTML first, ARIA only when necessary. If a semantic element already conveys the meaning, adding ARIA on top is redundant at best.

- **Common misuse** — sprinkling ARIA roles onto elements that already have native semantics, or adding a role without implementing the keyboard behavior that role implies (e.g. role="button" on a with no keyboard handler is worse than a real <button>).

Scope for this course: understand ARIA's purpose and recognize/apply basic roles — not the full specification.

Lecture 6 — Forms & Modern HTML

A. HTML Forms

- Purpose — the standard way a Web page collects input and sends it somewhere.

```
<form action="/submit" method="post">
  ...
</form>
```

Attribute	Meaning
action	The URL the form data is sent to.
method	How it's sent — GET or POST (see Section F).

Form controls (input, textarea, select, button) are the individual pieces a form is built from — covered next.

B. Input Elements

Every input type shares one element, <input>, distinguished by its type attribute:

type value	Used for
text	Single-line free text.
password	Text hidden as it's typed.
email	Email address, with built-in format validation.
number	Numeric input, often with a stepper.
tel	Phone number (no built-in format validation — use pattern).
url	A web address, with built-in format validation.
date / time / datetime-local	Native date and/or time pickers.
radio	One choice from a mutually exclusive set (shared name attribute).
checkbox	An independent on/off choice.

file	A file upload.
hidden	A value submitted with the form but not shown to the user.
submit / reset / button	Submits the form / resets it to defaults / a plain button with no default action.

C. Other Form Elements

Element	Role
<label>	Text description associated with a control (Lecture 5).
<textarea>	Multi-line free text input.
<select> / <option>	A dropdown, and its individual choices.
<optgroup>	Groups related <option>s under a shared sub-heading inside a <select>.
<button>	A clickable control; defaults to submitting the form unless type="button" is set.
<fieldset> / <legend>	Groups related controls visually and semantically, with a caption — e.g. grouping “Shipping Address” fields together.

D. Form Attributes

Attribute	Effect
name	The key the value is submitted under — required for a field's data to be sent at all.
value	The control's current or default value.
placeholder	Faint hint text shown when empty — not a substitute for a <label>.
required	Blocks submission until filled in.
disabled	Greyed out, not editable, and not submitted.
readonly	Visible and selectable, but not editable; still submitted.
checked / selected	Pre-selects a checkbox/radio, or an <option>.
multiple	Allows more than one selection (file input or <select>).
min / max	Numeric or date bounds.
minlength / maxlength	Text length bounds.
pattern	A regular expression the value must match.

E. HTML Form Validation

- **Validation is required** — catch obviously bad input (empty required field, malformed email) before it's sent anywhere.
- **HTML5 built-in validation** — the browser checks required, type="email", numeric constraints (min/max), length constraints (minlength/maxlength), and pattern automatically, with no JavaScript needed, and shows its own native validation messages.

Limitation: client-side validation can be disabled, bypassed, or the request can be sent directly with a tool like curl, entirely skipping the browser. Never trust it as a security boundary.

- **Why server-side validation is still required** — the server must re-validate everything client-side validation checked, because the client cannot be trusted. This is the direct setup for backend development in Week 8 — flag that connection explicitly to students.

F. Form Submission and HTTP

This is the lecture's payoff moment — connect Week 2 directly back to Week 1's HTTP model.

```
HTML Form
     ↓
HTTP Request
     ↓
  Server
```

Method	Behavior
GET	Form data is appended to the URL as query parameters — visible, bookmarkable, length-limited. Never use for passwords or sensitive data.
POST	Form data travels in the request body — not visible in the URL, no practical length limit. Used for anything that changes data or carries sensitive input.

Query parameters and request body were both introduced conceptually in Week 1 (Lecture 2's URL anatomy, and the HTTP request/response structure) — this is where they stop being abstract and become something students actually produce by submitting a form.

G. Embedding Multimedia

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  <source src="song.ogg" type="audio/ogg">
</audio>

<video width="480" controls poster="preview.jpg">
  <source src="clip.mp4" type="video/mp4">
  <track src="captions.vtt" kind="captions" srclang="en">
</video>
```

- **Multiple <source> elements** let the browser pick whichever format it supports — a fallback chain, not a single fixed file.
- **controls** shows the browser's native play/pause/volume UI; width/height sizes video the same way as images.
- **Captions/subtitles** — introduced via <track kind="captions">, tying directly back to the accessibility discussion in Lecture 5.

G. Embedding Multimedia

```
<iframe src="https://maps.example.com/embed" width="600" height="450"></iframe>
```

Used for embedding external content wholesale — maps, videos, documents — rather than rebuilding it.

> **Basic security consideration: an iframe embeds another site's content directly into yours, which is also the mechanism behind clickjacking attacks (an invisible iframe tricking a user into clicking something they didn't intend to). A sandbox attribute exists to restrict what an embedded frame can do — worth naming, full detail is out of scope for this course.**

H. Canvas

- **What is <canvas>?** A blank drawing surface element — it does nothing by itself.
- **Canvas vs. normal HTML elements** — ordinary elements are declarative markup; canvas is imperative — you draw onto it pixel by pixel using JavaScript, and the browser has no idea what's “on” it beyond a bitmap.

- **Canvas + JavaScript** — canvas is inert without script; getContext("2d") is the entry point to the Canvas API.
- **Use cases** — drawings, charts, games, data visualizations — anywhere pixel-level control matters more than DOM structure.

No detailed Canvas programming in this lecture — overview and recognition only. Students should know what it's for and reach for it later if a project needs it.

I. SVG

- **What is SVG?** Scalable Vector Graphics — an XML-based format that defines shapes mathematically rather than as pixels.
- **Vector vs. raster** — vector graphics (SVG) scale to any size with no quality loss; raster images (JPEG, PNG, canvas output) are fixed-resolution pixel grids that blur or pixelate when scaled up.
- **SVG in HTML** — can be embedded inline (<svg>...</svg> directly in the markup) or linked as an external .svg file, same as an image.
- **Basic SVG elements** — <svg>, <circle>, <rect>, <path> — named for recognition only, not authored by hand in this course.
- **SVG vs. Canvas** — SVG shapes remain individual, addressable DOM elements (can be styled with CSS, targeted with JavaScript, and stay crisp at any zoom level); canvas output is just pixels with no memory of how it got there.
- **Typical uses** — icons, diagrams, charts, logos — anywhere scalability and crispness matter more than pixel-level drawing control.

Overview only, same as Canvas — no hand-authoring of SVG paths in this course.

Tutorial — HTML Structure of the Semester Project

Brings together everything from Lectures 4–6. Students work through this sequence for their own project:

- Identify the project's major pages (e.g. Home, About, Login, Registration, Dashboard).
- Identify elements common to every page (header, nav, footer).
- Identify the navigation structure.
- Identify each page's main content area.
- Identify sections and articles within that content.
- Identify where forms are needed.
- Identify where tables or lists are appropriate.
- Identify images and media to include.
- Decide the semantic HTML element for each region — not “which div.”
- Consider accessibility for each page while designing it, not after.
- Produce a basic site map.
- Produce the HTML skeleton for at least one page.

Example site map:

Example per-page skeleton:

```
<header>
<nav>

<main>
  <section>
    <article>
      ...
    </article>
  </section>
</main>

<footer>
```

Lab — Semantic, Accessible Multi-Page Website

Part 1 — Project Structure

- Create the project directory.
- Organize HTML files.
- Organize images/assets into their own folder (per Week 1's folder-structure convention).
- Create navigation between pages.

Part 2 — HTML

- Build the semantic page structure for each page.
- Add headings and text content.
- Add links between pages.
- Add images.
- Add lists where appropriate.

Part 3 — Forms

- Create the project's forms.
- Use appropriate input types for each field.
- Add labels to every control.
- Add HTML5 validation attributes.
- Use required, pattern, and similar constraints deliberately, not by default.
- Test that the form actually submits.

Part 4 — Accessibility

- Add meaningful alt text to every image.
- Check heading hierarchy — no skipped levels.
- Ensure every form control has an associated label.
- Test keyboard navigation — Tab through the whole page.
- Inspect the semantic structure in DevTools and confirm it matches the intended hierarchy.

Part 5 — HTML Validation & Testing

- Inspect the HTML using browser DevTools.
- Identify and fix structural errors.
- Run the pages through the W3C HTML Validator (Week 1's tutorial).

- Test every link.
- Test every form.
- Test every page individually before considering the lab complete.